Antes de iniciar — Revisão obrigatória da BST

Antes de iniciar, abra seu código da BST e localize os seguintes elementos:

Elemento	O que você deve conseguir explicar
Classe No	Como um nó armazena o valor e as referências aos filhos
Classe BST	Como a raiz é armazenada
Inserção	Como o algoritmo decide entre esquerda e direita
Busca	Por que um dos ramos pode ser descartado
Recursividade	Como uma função pode processar subárvores
Percurso em ordem	Por que os valores aparecem ordenados
Altura	Como medir o maior caminho até uma folha

Se você não realizou a atividade anterior ou não consegue explicar esses componentes, deverá retomar o tutorial de BST e concluir essa implementação antes de prosseguir.

Tutorial de Implementação — Evoluindo uma BST para uma Árvore AVL

DSA | Estruturas de Dados e Algoritmos | Python | Estudo dirigido

Objetivo. Acrescentar balanceamento à Árvore Binária de Busca (BST) implementada anteriormente, sem descartar a estrutura já desenvolvida. Ao final, a árvore deverá atualizar alturas, detectar desbalanceamentos e executar rotações automaticamente após inserções.

Pré-requisito: a BST anterior deve estar funcional. Se você não compreende as classes No e BST, a inserção, as referências esquerda/direita e os percursos, retome a atividade anterior antes de avançar.

1. O que será mantido e o que será alterado

```
class No:
    def __init__(self, dado):
        self.dado = dado
        self.esquerda = None
        self.direita = None

class BST:
    def __init__(self):
        self.raiz = None
```

Componente anterior	Evolução necessária
Classe No	Acrescentar altura
Classe BST	Manter o nome ou renomear para AVL
Busca, mínimo, máximo e percursos	Reutilizar sem mudar a lógica
Inserção	Adaptar para recursão e rebalanceamento
Remoção	Adaptar em atividade posterior
Novos métodos	Altura, fator e rotações

Salve uma cópia do código original. O objetivo é observar as alterações feitas sobre sua própria implementação.

2. Etapa 1 — Acrescentar a altura aos nós

```
class No:
    def __init__(self, dado):
        self.dado = dado
        self.esquerda = None
        self.direita = None
        self.altura = 1
```

Neste tutorial, subárvore vazia tem altura 0; folha tem altura 1; um nó com filhos tem altura igual a 1 mais a maior altura dos filhos.

```
30  (altura 3)
/
20  (altura 2)
/
10  (altura 1)
```

3. Etapa 2 — Consultar e atualizar a altura

```
def altura(self, no):
    if no is None:
        return 0
    return no.altura


def atualizar_altura(self, no):
    altura_esquerda = self.altura(no.esquerda)
    altura_direita = self.altura(no.direita)
    no.altura = 1 + max(altura_esquerda, altura_direita)
```

A função altura consulta o atributo já armazenado. Não percorre a árvore para recalculá-lo. Após uma inserção, atualize primeiro os nós mais profundos, durante o retorno da recursão.

4. Etapa 3 — Calcular o fator de balanceamento

FB(n) = altura(esquerda) − altura(direita). Em uma AVL válida, cada nó deve ter FB igual a −1, 0 ou +1.

```
def fator_balanceamento(self, no):
    if no is None:
        return 0
    return self.altura(no.esquerda) - self.altura(no.direita)
```

FB	Interpretação
−1	Direita um nível mais alta
0	Alturas iguais
+1	Esquerda um nível mais alta
+2	Desbalanceamento à esquerda
−2	Desbalanceamento à direita

Verifique: na cadeia 30 → 20 → 10, ligada à esquerda, os fatores de 10, 20 e 30 são, respectivamente, 0, +1 e +2.

5. Etapa 4 — Compreender a rotação



A rotação reorganiza referências entre nós sem trocar os dados armazenados. Como a raiz da subárvore muda de 30 para 20, o método de rotação deve retornar a nova raiz.

6. Etapa 5 — Rotação à direita



A subárvore T2 precisa ser preservada. As outras subárvores, não representadas, permanecem ligadas aos mesmos nós.

```
def rotacao_direita(self, y):
    x = y.esquerda
    temp = x.direita

    x.direita = y
    y.esquerda = temp

    self.atualizar_altura(y)
    self.atualizar_altura(x)
    return x
```

Observe a ordem das atualizações: primeiro o nó que desceu (y), depois o nó que passou a ser a raiz (x).

7. Etapa 6 — Rotação à esquerda

```
Antes:              Depois:
    x                      y
     \                    /
      y                  x
     /                    \
    T2                    T2
```

```
def rotacao_esquerda(self, x):
    y = x.direita
    temp = y.esquerda

    y.esquerda = x
    x.direita = temp

    self.atualizar_altura(x)
    self.atualizar_altura(y)
    return y
```

As duas rotações são simétricas. Teste separadamente com as sequências 30, 20, 10 e 10, 20, 30.

8. Etapa 7 — Adaptar a inserção da BST

Se a inserção anterior era iterativa, desenvolva uma versão recursiva. O retorno recursivo permite atualizar as alturas dos ancestrais e substituir a raiz de uma subárvore após uma rotação.

```
def inserir_recursivo(self, no, dado):
    if no is None:
        return No(dado)

    if dado < no.dado:
        no.esquerda = self.inserir_recursivo(no.esquerda, dado)
    elif dado > no.dado:
        no.direita = self.inserir_recursivo(no.direita, dado)
    else:
        return no

    return no
```

Essa versão ainda é uma BST comum. Primeiro teste a inserção e o percurso em ordem. Depois, antes do retorno final, acrescente a atualização de altura e a verificação do fator de balanceamento.

```
self.atualizar_altura(no)
fb = self.fator_balanceamento(no)
```

9. Etapa 8 — Identificar os quatro casos

Caso	Sequência	Correção
LL	30, 20, 10	Rotação à direita
RR	10, 20, 30	Rotação à esquerda
LR	30, 10, 20	Esquerda no filho, depois direita
RL	10, 30, 20	Direita no filho, depois esquerda

```
# LL
if fb > 1 and dado < no.esquerda.dado:
    return self.rotacao_direita(no)

# RR
if fb < -1 and dado > no.direita.dado:
    return self.rotacao_esquerda(no)

# LR
if fb > 1 and dado > no.esquerda.dado:
    no.esquerda = self.rotacao_esquerda(no.esquerda)
    return self.rotacao_direita(no)

# RL
if fb < -1 and dado < no.direita.dado:
    no.direita = self.rotacao_direita(no.direita)
    return self.rotacao_esquerda(no)
```

Essas condições correspondem à inserção AVL apresentada neste tutorial. A remoção requer condições próprias.

10. Etapa 9 — Método completo de inserção

```
def inserir_recursivo(self, no, dado):
    # Inserção convencional da BST
    if no is None:
        return No(dado)

    if dado < no.dado:
        no.esquerda = self.inserir_recursivo(no.esquerda, dado)
    elif dado > no.dado:
        no.direita = self.inserir_recursivo(no.direita, dado)
    else:
        return no  # Duplicatas não são inseridas

    # Atualizar altura e verificar balanceamento
    self.atualizar_altura(no)
    fb = self.fator_balanceamento(no)

    # LL
    if fb > 1 and dado < no.esquerda.dado:
        return self.rotacao_direita(no)

    # RR
    if fb < -1 and dado > no.direita.dado:
        return self.rotacao_esquerda(no)

    # LR
    if fb > 1 and dado > no.esquerda.dado:
        no.esquerda = self.rotacao_esquerda(no.esquerda)
        return self.rotacao_direita(no)

    # RL
    if fb < -1 and dado < no.direita.dado:
        no.direita = self.rotacao_direita(no.direita)
        return self.rotacao_esquerda(no)

    return no


def inserir(self, dado):
    self.raiz = self.inserir_recursivo(self.raiz, dado)
```

Atenção: adapte ou substitua a inserção anterior. Não mantenha métodos inserir conflitantes. A atribuição a self.raiz é essencial porque uma rotação pode modificar a raiz da árvore.

11. Etapa 10 — Visualizar a árvore

```
def mostrar(self, no, nivel=0):
    if no is None:
        return

    self.mostrar(no.direita, nivel + 1)
    print(
        "    " * nivel + str(no.dado)
        + f" (h={no.altura}, FB={self.fator_balanceamento(no)})"
    )
    self.mostrar(no.esquerda, nivel + 1)
```

O método imprime o filho direito acima, o nó no centro e o filho esquerdo abaixo. Para 30, 20, 10, após o balanceamento:

```
    30 (h=1, FB=0)
20 (h=2, FB=0)
    10 (h=1, FB=0)
```

12. Etapa 11 — Testes obrigatórios

Inserções	Caso	Raiz esperada
30, 20, 10	LL	20
10, 20, 30	RR	20
30, 10, 20	LR	20
10, 30, 20	RL	20

Recrie uma árvore vazia para cada teste. Em todos, o percurso em ordem deve produzir 10 20 30.

```
arvore = BST()  # Ou AVL, conforme o nome adotado
for valor in [10, 20, 30, 40, 50, 25]:
    arvore.inserir(valor)
arvore.mostrar(arvore.raiz)
```

Estrutura esperada:

```
        30
       /  \
     20    40
    /  \     \
   10  25     50
```

A raiz deve ser 30 e a altura total, 3. O percurso em ordem deve produzir: 10 20 25 30 40 50.

13. Etapa 12 — Validar toda a árvore

```
def verificar_balanceamento(self, no):
    if no is None:
        return True

    fb = self.fator_balanceamento(no)
    if abs(fb) > 1:
        return False

    return (
        self.verificar_balanceamento(no.esquerda)
        and self.verificar_balanceamento(no.direita)
    )
```

Esse método verifica os fatores com base nas alturas armazenadas. Como extensão, recalcule as alturas pela estrutura para detectar também atributos desatualizados. Uma validação integral pode verificar ainda a propriedade de ordenação da BST.

14. Etapa 13 — Comparar BST e AVL

```
valores = list(range(1, 101))
```

Insira os valores em uma BST convencional e na AVL. A BST degenerada terá altura 100 (convenção de altura em nós). A AVL apresentada terá altura 7 nessa sequência. Compare também quantos nós são visitados ao buscar o valor 100.

Para refletir: a ordenação mudou? Os dados mudaram? Ou apenas a organização das referências? Por que o custo da busca é O(h) e a AVL garante h = O(log n)?

15. O que acontece com os métodos anteriores?

Busca, mínimo, máximo e percurso em ordem continuam válidos porque as rotações preservam a propriedade da BST. Se houver um método de remoção convencional, ele não deve ser reutilizado sem adaptação: remover também pode desbalancear a árvore. O escopo deste tutorial é a inserção autobalanceada.

16. Conclusão e preparação para árvores gerais

A evolução de BST para AVL acrescenta controle de altura, detecção de desbalanceamento e rotações. A estrutura permanece binária. Em árvores gerais, cada nó pode possuir múltiplos filhos; em estruturas especializadas como árvores B, mecanismos diferentes controlam a altura para garantir eficiência.

Resultado esperado: explicar com o próprio código como uma inserção provoca desbalanceamento, qual rotação corrige o problema, como as referências são reorganizadas e por que a ordenação permanece válida.